

Deep Q-Network



Outline

- **Why tabular Q-learning is not enough**
- Function approximation
- Deep Q-Network (DQN)
- Results and limitations



Why tabular Q-learning is not enough

- In tabular Q-learning, we store one value for each state-action pair
- This works well for small discrete problems

Why tabular Q-learning is not enough

- It becomes infeasible when:
 - the state is an image
 - the state is continuous
 - the number of states is enormous

We need a function approximator to generalize across states

Outline

- Why tabular Q-learning is not enough
- **Function approximation**
- Deep Q-Network (DQN)
- Results and limitations



Function Approximation

- Instead of storing a table, **we learn a function** $Q(s, a; \theta)$
 - Input: state (or state features)
 - Output: estimated Q-values
- Examples of **function approximators**:
 - neural networks
 - decision trees
 - other machine learning models

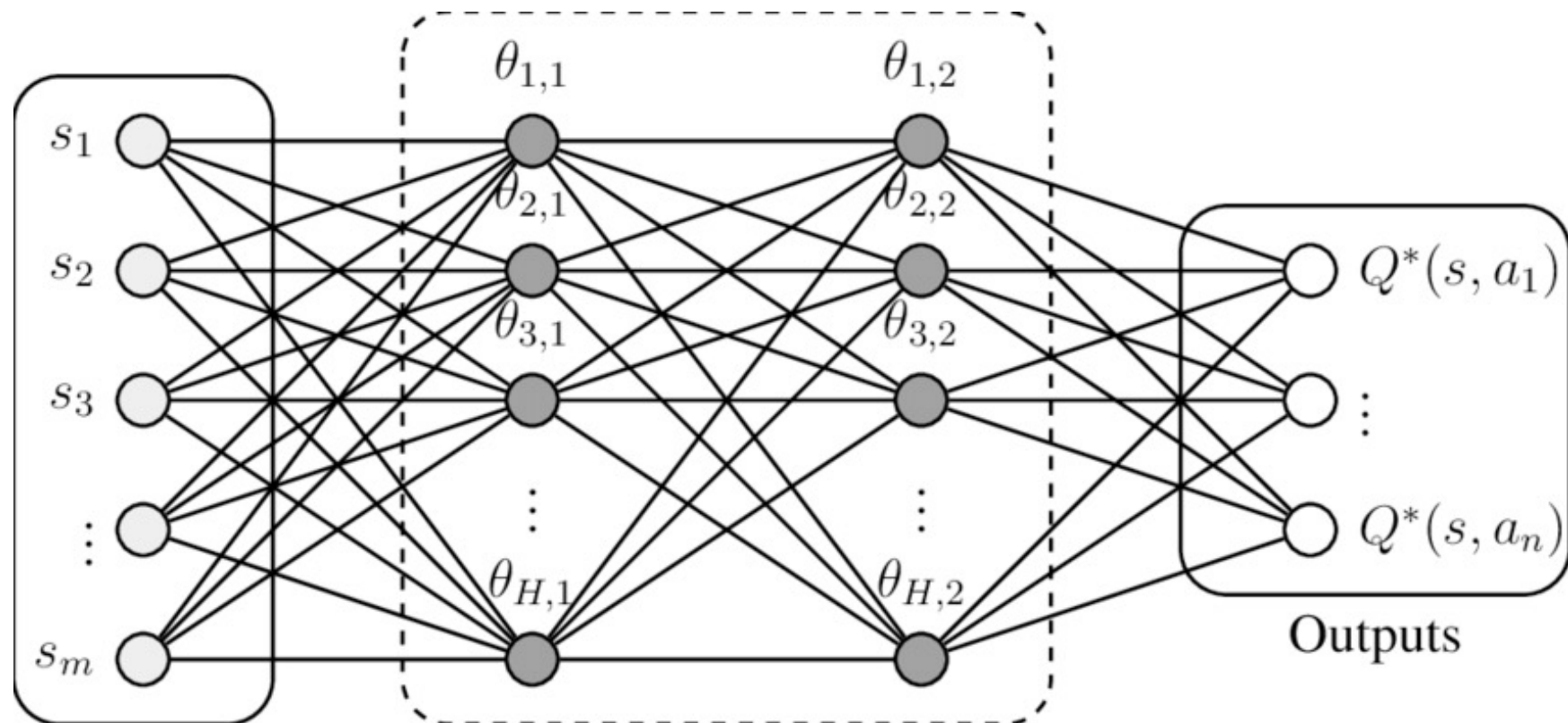
Function Approximation

- Function approximation plays a crucial role in **scaling up reinforcement learning**
- **Examples of applications:**
 - game-playing,
 - robotics,
 - autonomous systems.

Naive Q-learning with a neural network

- Replace the Q-table by a neural network
- The network receives the state as input
- The network outputs one Q-value per action

Naive Q-learning with a neural network



State = vector of features

Naive Q-learning with a neural network

- Represent $Q(s, a)$ by $Q(s, a; \theta)$
- For one transition (s, a, r, s') , define target:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta)$$

- Update θ to reduce:

$$(y - Q(s, a; \theta))^2$$

Why is this difficult?

- Problem: **unstable learning!**
- This happens because:
 - the target depends on the same parameters being updated
 - consecutive samples are highly correlated
 - the data distribution changes as the policy changes

Outline

- Why tabular Q-learning is not enough
- Function approximation
- **Deep Q-Network (DQN)**
- Results and limitations



Deep Q-Network

- Deep neural networks to **approximate the Q-function**
- It is useful when the state space is very large or high-dimensional
- Standard DQN assumes a **discrete action space**
- Two key ideas make DQN more stable:
 - experience replay,
 - target networks.

Experience Replay

- The agent stores transitions

$$e_t = (s_t, a_t, r_t, s_{t+1})$$

in a **replay memory** D

- During training, we **sample random minibatches** from D

Experience Replay

- **Why it helps:**
 - breaks the correlations between consecutive samples
 - reuses past experience
 - makes training more stable

Target Network

- DQN keeps two networks:
 - **online network** $Q(s, a; \theta)$
 - **target network** $Q(s, a; \theta^-)$
- **The target network is updated only every C steps**

Target Network

- **Why it helps:**
 - the target changes more slowly
 - this reduces instability during training

Deep Q-networks



<https://www.youtube.com/watch?v=wrBUkpiRvCA>

Deep Q-Network

LETTER

doi:10.1038/nature14236

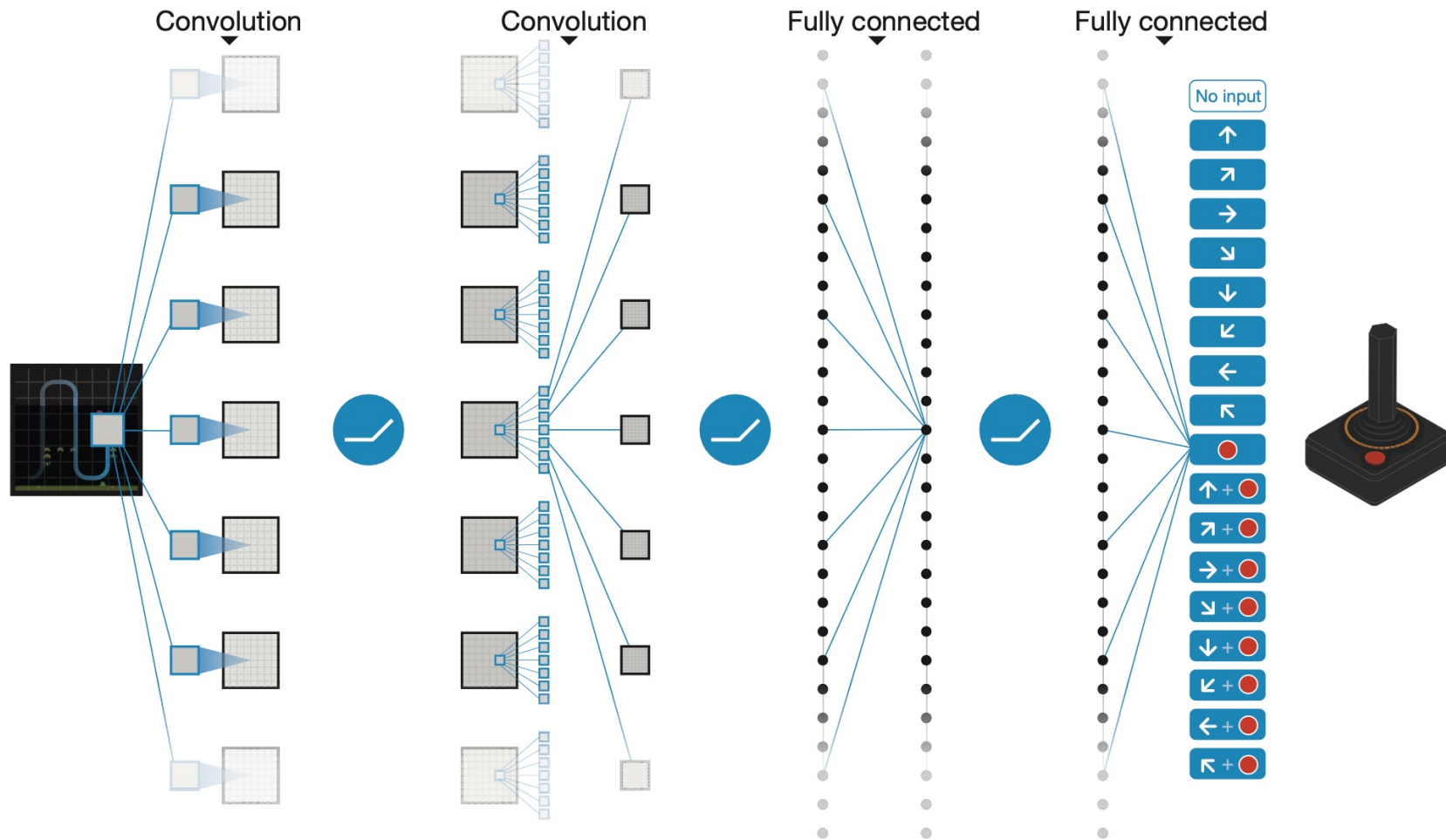
Human-level control through deep reinforcement learning

Volodymyr Mnih^{1*}, Koray Kavukcuoglu^{1*}, David Silver^{1*}, Andrei A. Rusu¹, Joel Veness¹, Marc G. Bellemare¹, Alex Graves¹, Martin Riedmiller¹, Andreas K. Fidjeland¹, Georg Ostrovski¹, Stig Petersen¹, Charles Beattie¹, Amir Sadik¹, Ioannis Antonoglou¹, Helen King¹, Dhharshan Kumaran¹, Daan Wierstra¹, Shane Legg¹ & Demis Hassabis¹

The theory of reinforcement learning provides a normative account¹, deeply rooted in psychological² and neuroscientific³ perspectives on animal behaviour, of how agents may optimize their control of an

agent is to select actions in a fashion that maximizes cumulative future reward. More formally, we use a deep convolutional neural network to approximate the optimal action-value function

Deep Q-Network



Deep Q-Network

- DQN's loss function

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{U}(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i) \right)^2 \right]$$

- Q-Learning

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

TD Error

Deep Q-Network

- DQN's loss function

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim U(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i) \right)^2 \right]$$



*Parameters of the
Target Network
(updated every C steps)*

Deep Q-Network

- DQN's loss function

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{U}(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i) \right)^2 \right]$$



*Batch sampled from **experience replay***

*Agent's experience $e_t = (s_t, a_t, r_t, s_{t+1})$
is stored in data set D_t*

Deep Q-Network

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

 With probability ε select a random action a_t

 otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

 Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

 Every C steps reset $\hat{Q} = Q$

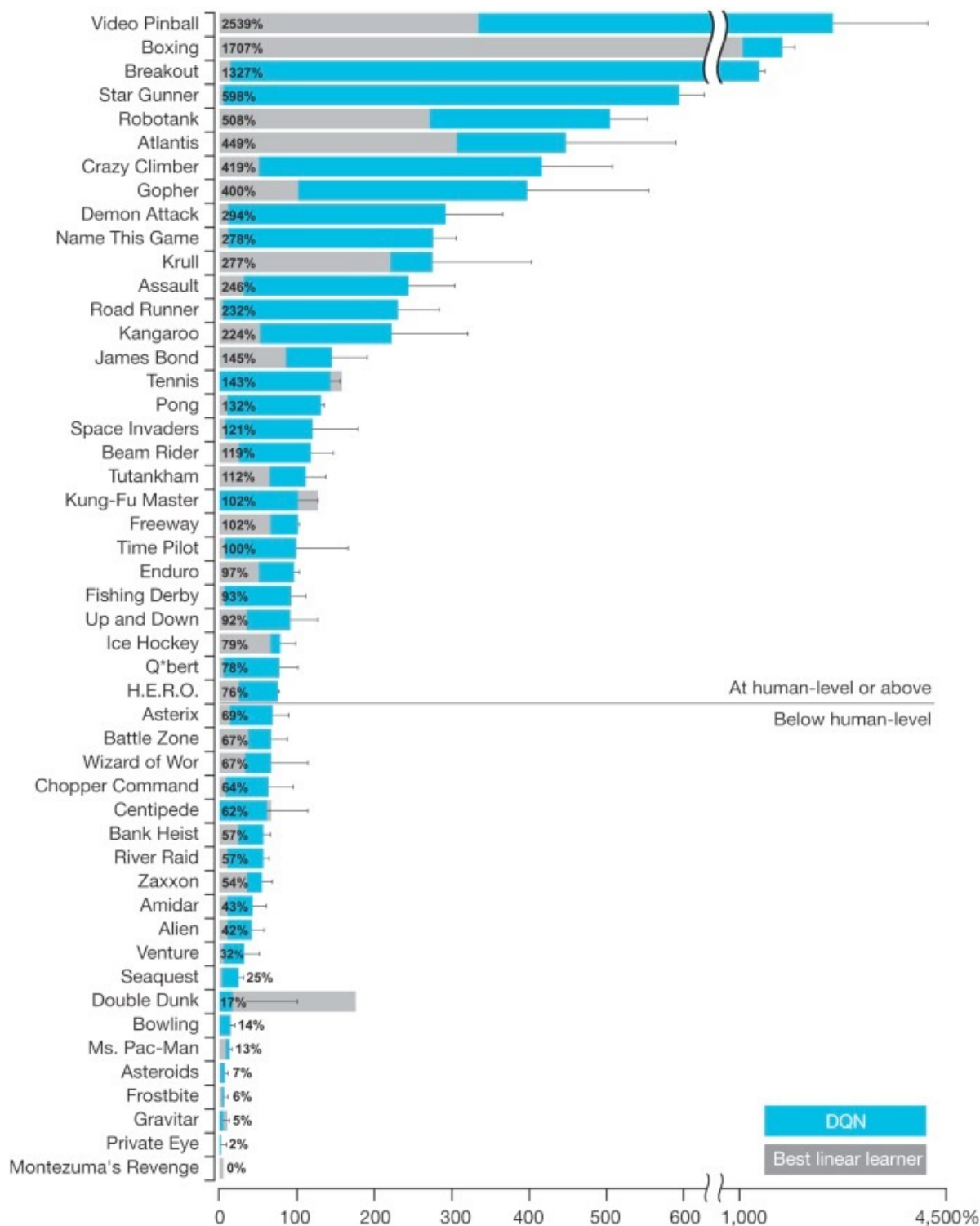
End For

End For

Outline

- Why tabular Q-learning is not enough
- Function approximation
- Deep Q-Network (DQN)
- **Results and limitations**





- DQN learned directly from high-dimensional visual input
- The same algorithm was applied to many Atari games
- It achieved human-level or better performance in several cases

Limitations of basic DQN

- Only works naturally with **discrete actions**
- Can be **unstable and sensitive to hyperparameters**
- **Sample inefficient**
- Training can be **slow**
- **Later improvements include:**
 - **Double DQN** improves the **target calculation**
 - **Prioritized Replay** improves the **sampling of experience**

Thank You



sardinha@inf.puc-rio.br